

Relazione Progetto Lab. Applicazioni Mobili

Ultiwear

Realizzata da Alessandro Nanni

Email: alessandro.nanni17@studio.unibo.it

Matricola: 0001027757

Indice

1	Overview dell'app	1
1.1	Note tecniche	1
1.2	Primo avvio dell'app	1
2	Guardaroba/Wardrobe	2
3	Esplora/Browse	3
4	Eventi/Upcoming Events	3

1 Overview dell'app

L'applicazione Ultiwear è progettata per giocatori di frisbee con la passione per lo scambio di capi d'abbigliamento.

Tramite essa è possibile visualizzare i propri capi in un armadio digitale, pubblicarli, mostrare interesse per eventuali scambi, consultare i tornei futuri e parteciparvi, e gestire scambi in tempo reale.

Dato che l'app non vuole sostituire il momento dello scambio in persona, e lasciare un pò di mistero e magia, non ci sono username pubblici e non si ha modo di comunicare con gli altri.

1.1 Note tecniche

Quasi tutti i dati utilizzati sono salvati in un database *firebase*: questo include sia le rappresentazioni in forma di tabelle di oggetti, che le immagini.

L'app è realizzata con *jetpack compose*: non ci sono file XML contenenti *view* nella cartella *res*.

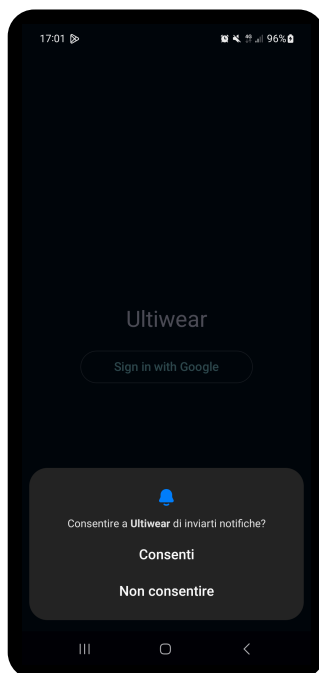
L'autenticazione è effettuata tramite account Google.

Come consigliato dalla [documentazione ufficiale](#), l'app utilizza una singola attività, la navigazione tra le diverse sezioni è gestita tramite lo stato di Compose.

Si cerca di rispettare il modello MVVM, sono presenti *package* con compiti precisi:

- *data*: comunicazione con il database;
- *model*: contiene le *data class*
- *notifications*: si occupa dell'invio di notifiche;
- *view*: contiene i *composable*;
- *viewModel*: contiene i *ViewModel* intermediari tra *View* e *Model*

1.2 Primo avvio dell'app



La prima volta che si avvia l'app verranno richiesti i permessi necessari: invio di notifiche, posizione e accesso alla fotocamera, richiesti dal manifest.

```
AndroidManifest.xml
1  <uses-permission
    android:name="android.permission.CAMERA" />
2  <uses-permission
    android:name="android.permission.POST_NOTIFICATIONS"/
    >
3  <uses-permission
    android:name="android.permission.ACCESS_FINE_LOCATION
    >
```

Nella *MainActivity*, questi vengono richiesti in catena affinché l'ultimo (*locationPermission*), non sovrascriva i precedenti.

```

1 private fun requestPermissionsOnStartup() {
2     notificationPermissionLauncher.launch(POST_NOTIFICATIONS)
3 }
4 private val notificationPermissionLauncher =
5     registerForActivityResult(ActivityResultContracts.RequestPermission()) { _ ->
6         requestCameraPermission() // next
7     }

```

Listing 1: Dopo aver richiesto il permesso per le notifiche, si chiederà quello per la fotocamera

Successivamente, tramite stati *composable* dell' `authViewModel`, si controlla che l'utente sia registrato e connesso a internet.



In base a questi stati si mostreranno 3 *view* diverse. Dopo aver eseguito l'accesso con Google, l'utente potrà utilizzare l'app.

```

1 GetCredentialRequest.Builder().addCredentialOption(
2     GetGoogleIdOption.Builder()
3         .setFilterByAuthorizedAccounts(false)
4         .setServerClientId(BuildConfig.GOOGLE_CLIENT_ID)
5         .setAutoSelectEnabled(false)
6         .build()).build()

```

Listing 2: Codice per creare il prompt “sign in with Google” e ottenere un token di autenticazione

Il token di login sarà poi convertito in un token Firebase, memorizzato in una collezione come UID assieme alla mail dell'utente. Ora l'utente ha accesso all'app, e potrà navigare le sue 5 *view*.

`var selectedIndex by rememberSaveable { mutableIntStateOf(0) }` tiene traccia della *sub-activity* corrente. Ognuna di esse è definita dal *composable* `TabItem`.

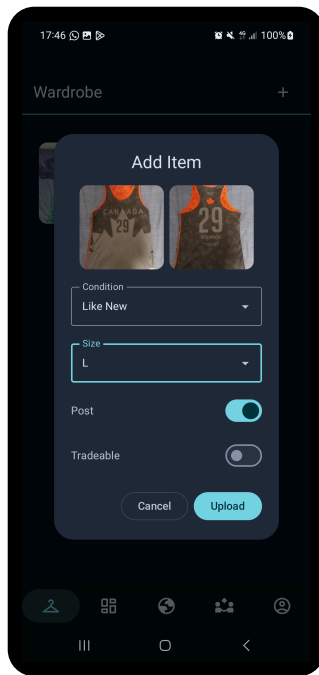
```

1 val tabs = listOf(
2     TabItem(
3         "Wardrobe",
4         R.drawable.wardrobe
5     ) {
6         WardrobeScreen(wardrobeViewModel)
7     },
8     ...
9 )

```

2 Guardaroba/Wardrobe

Qui l'utente può inserire dati e foto relativi ai suoi capi d'abbigliamento.

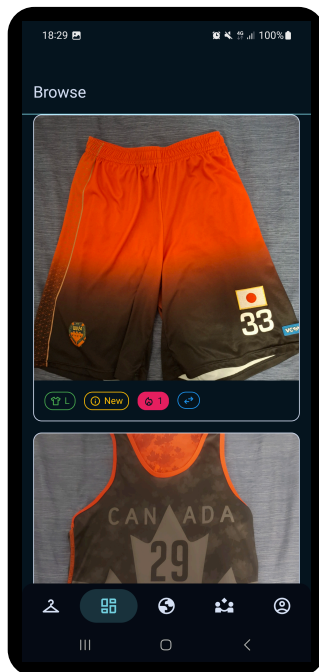


Per ogni capo si può fare una foto del fronte, retro, specificare taglia e condizioni, e infine scegliere se pubblicare¹ e indicare se questo capo lo si vuole scambiare. I capi sono mostrati tramite una `LazyVerticalGrid` di 3 colonne. Cliccando un elemento della griglia si potranno visualizzare i dettagli del capo.

Le operazioni sul database sono effettuate tramite l'oggetto `ItemHandler`, di cui il `WardrobeViewModel` possiede un riferimento. Un listener controlla continuamente cambiamenti nella collezione wardrobe per gli oggetti dell'owner. In caso di cambiamenti si aggiornano 3 liste di item: una che li contiene tutti, una che contiene solo quelli scambiabili e un'ultima che contiene solo quelli posseduti (`items.filter { it.owned }`). Il listener permette di aggiornare automaticamente gli *item* dell'utente in ogni punto dell'app (*Wardrobe*, *Browse* e *Trade*).

Ogni oggetto capo contiene la variabile booleana `owned`. Questa è usata per indicare se il capo deve essere mostrato nell'armadio, nel caso l'utente corrente non ne sia più il proprietario in seguito ad uno scambio².

3 Esplora/Browse



In questa sezione si possono vedere le foto e dettagli degli *item* postati. Per ogni post non proprio si può anche mettere like e se il proprietario lo ha permesso, esprimere interesse di scambio. I bottoni di scambio e like sono oscurati e non cliccabili per i propri capi d'abbigliamento. Ogni post è contenuto in una `LazyColumn`, ed è una `Card` composta da un `Pager` che si può scorrere per vedere le foto di fronte e retro e una `InfoBar`.

Il `BrowseViewModel` contiene un riferimento allo stesso handler, e dunque listener del di *Wardrobe*. Dunque, quando un utente preme il bottone like, il `ViewModel` viene notificato di questo cambiamento e ricarica la colonna. Per evitare che un utente possa mettere like allo stesso post più volte, ogni post dispone di una collezione che contiene gli id degli utenti che hanno messo like.

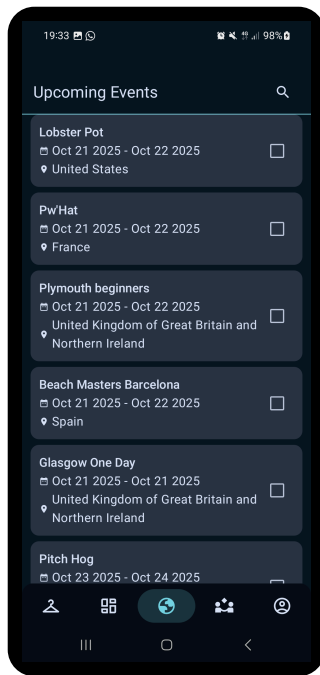
Similmente, la collezione `trade_interests` tiene traccia degli utenti che hanno espresso interesse a scambiare un certo item.

4 Eventi/Upcoming Events

Qui si possono vedere i prossimi tornei da tutto il mondo, ottenuti tramite l'api di [Ultical](#). Sono *card* ordinate dalla data di inizio più vicina alla più lontana. Ogni card ha associata una *checkbox* per indicare se si sarà presenti ad un certo torneo.

¹Visibile nella sezione *Browse*, mostrata in seguito.

²Eseguibile nella sezione *Quick Trade*, mostrata in seguito.



Il singleton `APIClient` inizializza una sola volta (tramite `by lazy`) l'interfaccia `UlticalAPI` fornendogli l'url a cui fare la chiamata HTTP, e una factory da usare per processare i dati. In questo caso si usa la libreria GSON di Google in quanto l'API di `ultical` restituisce un oggetto JSON.

```
1 val api: UlticalApi by lazy {  
2     Retrofit.Builder()  
3         .baseUrl(URL)  
4         .addConverterFactory(  
5             GsonConverterFactory.create()  
6         )  
7         .build()  
8         .create(UlticalApi::class.java)  
9 }
```

L'interfaccia fornisce un metodo `getEvents()` per ottenere una lista di eventi.